

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет информатика и системы управления
Кафедра системы обработки информации и управления

Курс «Парадигмы и конструкции языков программирования»

Отчет по рубежному контролю №2

Вариант А31

Выполнил:
студент группы ИУ5-32Б:
Воронков А.Д.
Подпись и дата:

Проверил:
преподаватель каф. ИУ5
Гапанюк Ю.Е.
Подпись и дата:

Москва, 2025 г.

Текст программы

Файл rk2.py

```
class DataTable:
    def __init__(self, id, name, row_count, db_id):
        self.id = id
        self.name = name
        self.row_count = row_count
        self.db_id = db_id

class Database:
    def __init__(self, id, name):
        self.id = id
        self.name = name

class TableDatabase:
    def __init__(self, db_id, table_id):
        self.db_id = db_id
        self.table_id = table_id

def initialize_data():
    databases = [
        Database(1, 'производственная база данных'),
        Database(2, 'архивная база'),
        Database(3, 'тестовая база данных'),
        Database(4, 'другая производственная БД'),
        Database(5, 'архивная база (копия)'),
        Database(6, 'тестовая БД данных'),
    ]

    tables = [
        DataTable(1, 'пользователи', 15000, 1),
        DataTable(2, 'заказы', 85000, 2),
        DataTable(3, 'продукты', 25000, 3),
        DataTable(4, 'покупатели', 35000, 3),
        DataTable(5, 'сотрудники', 5000, 3),
    ]

    tables_databases = [
        TableDatabase(1, 1),
        TableDatabase(2, 2),
        TableDatabase(3, 3),
        TableDatabase(3, 4),
        TableDatabase(3, 5),
        TableDatabase(4, 1),
        TableDatabase(5, 2),
        TableDatabase(6, 3),
        TableDatabase(6, 4),
        TableDatabase(6, 5),
    ]

    return databases, tables, tables_databases

def build_one_to_many(databases, tables):
    return [(t.name, t.row_count, d.name)
            for d in databases]
```

```

        for t in tables
        if t.db_id == d.id]

def build_many_to_many(databases, tables, tables_databases):
    many_to_many_temp = [(d.name, td.db_id, td.table_id)
                          for d in databases
                          for td in tables_databases
                          if d.id == td.db_id]

    many_to_many = [(t.name, t.row_count, db_name)
                    for db_name, db_id, table_id in many_to_many_temp
                    for t in tables if t.id == table_id]

    return many_to_many

def query1_sorted_tables(databases, tables):
    one_to_many = build_one_to_many(databases, tables)
    return sorted(one_to_many, key=lambda i: i[2])

def query2_databases_with_row_sum(databases, tables):
    one_to_many = build_one_to_many(databases, tables)

    res2_unsorted = []
    for d in databases:
        d_tables = list(filter(lambda i: i[2] == d.name, one_to_many))
        if len(d_tables) > 0:
            d_rows = [rows for _, rows, _ in d_tables]
            d_rows_sum = sum(d_rows)
            res2_unsorted.append((d.name, d_rows_sum))

    return sorted(res2_unsorted, key=lambda i: i[1], reverse=True)

def query3_databases_with_base_in_name(databases, tables, tables_databases):
    many_to_many = build_many_to_many(databases, tables, tables_databases)

    res3 = {}
    for d in databases:
        if 'база' in d.name.lower():
            d_tables = list(filter(lambda i: i[2] == d.name, many_to_many))
            d_tables_names = [x for x, _, _ in d_tables]
            res3[d.name] = d_tables_names

    return res3

def print_results(res1, res2, res3):
    print('Запрос 1')
    print('Список всех связанных таблиц и баз данных, отсортированный по базам данных:')
    for i in res1:
        print(i[2] + ": " + i[0] + ", строки: " + str(i[1]))

    print('\nЗапрос 2')
    print('Список баз данных с суммарным количеством строк таблиц в каждой БД:')
    for i in res2:
        print(i[0] + ": " + str(i[1]) + " строк")

    print('\nЗапрос 3')
    print('Список всех баз данных, у которых в названии присутствует слово "база", и список таблиц

```

```

в них:')
    for db_name, table_list in res3.items():
        print(db_name + ": " + ", ".join(table_list))

def main():
    databases, tables, tables_databases = initialize_data()

    res1 = query1_sorted_tables(databases, tables)
    res2 = query2_databases_with_row_sum(databases, tables)
    res3 = query3_databases_with_base_in_name(databases, tables, tables_databases)

    print_results(res1, res2, res3)

if __name__ == '__main__':
    main()

```

Файл test_rk2.py

```

import unittest

from rk2 import (
    DataTable, Database, TableDatabase,
    initialize_data, build_one_to_many, build_many_to_many,
    query1_sorted_tables, query2_databases_with_row_sum,
    query3_databases_with_base_in_name
)

class TestDatabaseQueries(unittest.TestCase):

    def setUp(self):
        """Подготовка тестовых данных перед каждым тестом"""
        self.databases = [
            Database(1, 'производственная база данных'),
            Database(2, 'архивная база'),
            Database(3, 'тестовая база данных'),
            Database(4, 'другая производственная БД'),
            Database(5, 'архивная база (копия)'),
            Database(6, 'тестовая БД данных'),
        ]

        self.tables = [
            DataTable(1, 'пользователи', 15000, 1),
            DataTable(2, 'заказы', 85000, 2),
            DataTable(3, 'продукты', 25000, 3),
            DataTable(4, 'покупатели', 35000, 3),
            DataTable(5, 'сотрудники', 5000, 3),
        ]

        self.tables_databases = [
            TableDatabase(1, 1),
            TableDatabase(2, 2),
            TableDatabase(3, 3),
            TableDatabase(3, 4),
            TableDatabase(3, 5),
            TableDatabase(4, 1),
        ]

```

```
TableDatabase(5, 2),
TableDatabase(6, 3),
TableDatabase(6, 4),
TableDatabase(6, 5),
]
```

```
def test_build_one_to_many(self):
    """Тест 1: Проверка построения отношения один-ко-многим"""
    result = build_one_to_many(self.databases, self.tables)

    self.assertEqual(len(result), 5)

    for table_name, row_count, db_name in result:
        self.assertIsInstance(table_name, str)
        self.assertIsInstance(row_count, int)
        self.assertIsInstance(db_name, str)

    db_names = [db_name for _, _, db_name in result]
    self.assertIn('производственная база данных', db_names)
    self.assertIn('архивная база', db_names)
    self.assertIn('тестовая база данных', db_names)

def test_query1_sorted_tables(self):
    """Тест 2: Проверка запроса 1 - сортировка таблиц по именам баз данных"""
    result = query1_sorted_tables(self.databases, self.tables)

    db_names = [db_name for _, _, db_name in result]
    sorted_db_names = sorted(db_names)
    self.assertEqual(db_names, sorted_db_names)

    self.assertEqual(len(result), 5)

    self.assertEqual(result[0][2], 'архивная база')

def test_query3_databases_with_base_in_name(self):
    """Тест 3: Проверка запроса 3 - поиск БД с 'база' в названии"""
    result = query3_databases_with_base_in_name(
        self.databases, self.tables, self.tables_databases
    )

    self.assertIsInstance(result, dict)

    for db_name in result.keys():
        self.assertIn('база', db_name.lower())

    expected_dbs = ['архивная база', 'архивная база (копия)',
                    'производственная база данных', 'тестовая база данных']

    for db_name in expected_dbs:
        self.assertIn(db_name, result)

    self.assertNotIn('другая производственная БД', result)
    self.assertNotIn('тестовая БД данных', result)

    for db_name, tables_list in result.items():
        self.assertIsInstance(tables_list, list)
```

```

def test_initialize_data_function(self):
    """Дополнительный тест: Проверка функции инициализации данных"""
    databases, tables, tables_databases = initialize_data()

    self.assertEqual(len(databases), 6)
    self.assertEqual(len(tables), 5)
    self.assertEqual(len(tables_databases), 10)

    self.assertIsInstance(databases[0], Database)
    self.assertIsInstance(tables[0], DataTable)
    self.assertIsInstance(tables_databases[0], TableDatabase)

if __name__ == '__main__':
    unittest.main(verbosity=2)

```

Скриншот работы приложения

```

● alex_voronkov@LAPTOP-08AG1BBP:~/3sem$ python3 test_rk2.py
test_build_one_to_many (__main__.TestDatabaseQueries.test_build_one_to_many)
Тест 1: Проверка построения отношения один-ко-многим ... ok
test_initialize_data_function (__main__.TestDatabaseQueries.test_initialize_data_function)
Дополнительный тест: Проверка функции инициализации данных ... ok
test_query1_sorted_tables (__main__.TestDatabaseQueries.test_query1_sorted_tables)
Тест 2: Проверка запроса 1 - сортировка таблиц по именам баз данных ... ok
test_query3_databases_with_base_in_name (__main__.TestDatabaseQueries.test_query3_databases_with_base_in_name)
Тест 3: Проверка запроса 3 - поиск БД с 'база' в названии ... ok

-----
Ran 4 tests in 0.001s

OK

```

Рис. 1. ВЫВОД РЕЗУЛЬТАТОВ ТЕСТОВ